CPE 2300 · RISC-V · TERM PROJECT

Project Workbook

Set your **plan** once. Then each Wednesday has its own page where your two goals appear automatically — record how much you attained, any plan changes, each member's contribution, the core code you wrote, and the challenges you hit. Final deliverables (one code file + report) are due **Friday, December 4 at 11:59 PM.**

PLAN

Project Lock-in

set once · objectives are frozen after this

Project title

Conway's Game of Life

Team name

Team members — up to 3; each gets their own space on every weekly page

Will Bakey	wjbakey@gmail.com
Logan Chook	loganchook@gmail.com
Daiki Hew	tylerhew@gmail.com

Objectives — 2 to 3 core objectives (must-have), then add-on features (if time allows); give each a deadline Wednesday

CORE OBJECTIVES — MUST-HAVE · 80% OF PRODUCT GRADE

Implement game rules and logic to simulate successive generations	Wed Nov 11
Display the simulation on the VGA monitor, updating it with each ge	Wed Nov 18
Implement predefined patterns that can be selected and allow step-	Fri Dec 4

ADD-ON FEATURES — IF TIME ALLOWS · 20%

Allow users to manually toggle cells to create custom simulations	Fri Dec 4
Add automatic simulation and the ability to pause it	Fri Dec 4
Add reset function to revert simulation, and a clear function to rese	Fri Dec 4

Two goals per Wednesday — link each to a main objective above; Thanksgiving week (Nov 25) is skipped

WED NOV 4	WED NOV 11	WED NOV 18
Research arrays, buffer	Implement grid initiali	Display the grid using V
Core 1: Implement game	Core 1: Implement game	Core 2: Display the simu

Design program structu	Implement and test one	Implement basic keybo
Core 1: Implement game	Core 1: Implement game	Core 3: Implement prede

Work division — three owners: name + what they own

Will Bakey	Week 1 - 2D grid representation, array indexing, and initialization Week 2 - Drawing individual cells on the VGA
Logan Chook	Week 1 - Neighbor counting and boundary checks Week 2 - Full-grid rendering routine Week 3 - Keyboard input step-through simulation
Daiki Hew	Week 1 - Game of Life rules, generation updates, and double buffering Week 2 - Game logic with VGA output, testing successive

Function skeleton — function names + a one-line purpose each

initialize_grid — Initialize the game grid with a selected starting pattern.

count_neighbors — Count the number of living neighbors surrounding a specified cell.

update_generation — Apply the Game of Life rules to calculate the next generation.

swap_buffers — Exchange the current and next grid buffers after each generation.

draw_grid — Display the current grid on the VGA framebuffer.

handle_keyboard — Read keyboard input to select starting patterns or advance the simulation by one generation.

main_loop — Coordinate keyboard input, generation updates, and VGA output.

Block / system diagram link

https://github.com/wjbakey/CPE2300_Final

GitHub repo URL

https://github.com/wjbakey/CPE2300_Final

ACADEMIC INTEGRITY

We affirm that all RISC-V **assembly code** submitted for this project is written by our team members ourselves. We understand that using **AI tools** (for example ChatGPT, Claude, or GitHub Copilot) to generate assembly code is a **violation of Cal Poly's Academic Integrity Policy** and will be reported as such.

- [x] **We have read and agree to the statement above.**

Signed — type the full names of all members

Will Bakey, Logan Chook, Daiki Hew

Date

11/05/2026

- [x] We understand these **goals are locked** after the plan. Any change is recorded on that week's page under "Changes to the plan."

TEST PLAN

Demo & Test Checklist

complete before the plan is locked

For every core objective and add-on, define how you will prove that it works. This becomes the approved checklist for your final demo and evaluation.

A useful test names the setup, exact input/action, and expected observable output. Do not write only "it works."

Core objective 1: Implement game rules and logic to simulate successive generations

Write the agreed test now. This is what you will demonstrate at the end.

Success criterion

The program correctly calculates successive generations using Conway's Game of Life rules, neighbor counting, and double

Owner(s)

Will, Logan, Daiki

How will it be tested?

Initialize known patterns and compare the calculated generations against their expected states using console output / memory inspection.

Exact test inputs / actions

Initialize a 2×2 block and advance three generations. Then initialize a three-cell blinker and advance two generations. Hardcoded in the program for testing purposes

Expected observable result

The block remains unchanged after all three generations. The blinker changes from vertical to horizontal after one generation, then returns to vertical after two generations

Evidence to bring to the final demo

Console outputs / memory screenshots showing the initial / resulting generations, GitI

Core objective 2: Display the simulation on the VGA monitor, updating it with each generation

Write the agreed test now. This is what you will demonstrate at the end.

Success criterion

The VGA display correctly represents the current grid, showing living and dead cells in different colors and updating after each generation.

Owner(s)

Will, Logan, Daiki

How will it be tested?

Display a predefined pattern on the VGA monitor and compare the displayed cells with the expected grid. Advance the simulation and

Exact test inputs / actions

Load a three-cell blinker pattern, display it on the VGA monitor, and advance one generation. Hardcoded

Expected observable result

The initial three vertical living cells appear in the correct locations. After advancing, three horizontal living cells appear in their place, with all other cells displayed as dead

Evidence to bring to the final demo

VGA screenshots before / after one generation, GitHub commit.

Core objective 3: Implement predefined patterns that can be selected and allow step-through simulation

Write the agreed test now. This is what you will demonstrate at the end.

Success criterion

The user can select predefined starting patterns and advance the simulation one generation at a time using keyboard controls

Owner(s)

Will, Logan, Daiki

How will it be tested?

Use keyboard inputs to select different patterns and manually advance the simulation

Exact test inputs / actions

Press 1 to load a block, 2 to load a blinker, and 3 to load a glider. After loading the blinker, press N twice, releasing the key between presses

Expected observable result

Each number key loads its corresponding starting pattern. The N key advances exactly one generation per press. The blinker returns to its initial orientation after two presses, and the simulation does not advance without additional input

Evidence to bring to the final demo

VGA screenshots of selected patterns, before/after N is pressed each time, GitHub com

Add-on 1: Allow users to manually toggle cells to create custom simulations

Write the agreed test now. This is what you will demonstrate at the end.

Success criterion

The user can modify individual cells by clicking on them with the mouse before advancing the simulation.

Owner(s)

Will, Logan, Daiki -- (time permitting)

How will it be tested?

Use the mouse to create and modify a small pattern, then advance the simulation

Exact test inputs / actions

Load a starting pattern, click a dead cell to make it alive, then click the same cell again to make it dead. Create a three-cell blinker using mouse clicks and advance one

Expected observable result

Each distinct click toggles only the selected cell. The blinker created using the mouse changes orientation correctly after advancing

Evidence to bring to the final demo

VGA screenshots showing before/after mouse editing, live demo of feature, GitHub cor

Add-on 2: Add automatic simulation and the ability to pause it

Write the agreed test now. This is what you will demonstrate at the end.

Success criterion

The simulation automatically advances through generations at a controlled speed and can be started or paused using keyboard input

Owner(s)

Will, Logan, Daiki -- (time permitting)

How will it be tested?

Start automatic simulation using a known pattern, observe multiple generation updates, then pause the simulation

Exact test inputs / actions

Load a blinker by pressing 2, press Space to start automatic simulation, wait for several generations, and press Space again to pause

Expected observable result

The blinker repeatedly alternates between vertical and horizontal orientations without additional keypresses. Once paused, the displayed grid stops changing

Evidence to bring to the final demo

Short video or live demo showing automatic updates and pause functionality, GitHub c

Add-on 3: Add reset function to revert simulation, and a clear function to reset board

Write the agreed test now. This is what you will demonstrate at the end.

Success criterion

The user can press a key to revert the simulation back to the initialized generation, or press a key to clear the board to all dead cells

Owner(s)

Will, Logan, Daiki -- (time permitting)

How will it be tested?

Exact test inputs / actions

Initialize cells to be simulated, let it run a few generations, then press the reset key. Run the simulation for a few generations again, then press the clear key

Press 3 to load a glider. Press N 3 times to step through generations. Press R to reset. Press N 3 times to step through generations. Press C to clear the board.

Expected observable result

The glider should move across the grid slightly. Pressing R will move it back to its original position at its original stage. The glider will move across again, the pressing C will create a blank board.

Evidence to bring to the final demo

Screenshots of initial stage, before/after pressing R, before/after pressing C, GitHub co

CPE 2300 · RISC-V Term Project. Entries are stored in this browser only; print each week to a PDF to submit.